

DEFRAL — Product Requirements Document

Pinjaman ter-collateral yang membela dirinya sendiri, dengan agent yang bisa dibuktikan tidak bisa berbuat apa-apa lagi.

Status	FROZEN — 2026-08-10
Owner dokumen	alven
Kontributor	alven — smart contract · islakun — backend & agent · bima — frontend & video
Target	KeeperHub Agents Onchain Hackathon (DoraHacks)
Deadline	2026-08-13 · 12:00 UTC+2 · 17:00 WIB
Chain	Base Sepolia 84532
Repo	github.com/alventendrawan123/defral
Plan eksekusi	SC/plan.md · BE/plan.md · FE/plan.md
Predecessor	Cermin-RWA — Build on Canton (Encode Club), Juli 2026
Alamat kontrak	(diisi Senin 11 Agt)
Tautan bukti	(diisi Selasa 12 Agt)

Dokumen ini menjawab **APA** dan **KENAPA**. Detail GIMANA ada di tiga `plan.md`. Kalau keduanya bertentangan, `plan.md` yang menang untuk signature dan endpoint; PRD yang menang untuk perilaku dan batasan.

BAGIAN I — KENAPA

1. WRITE-UP

Ini halaman yang di-copy ke DoraHacks.

Kalau Anda meminjam dengan jaminan aset, Anda hidup di atas satu angka: harga jaminan Anda. Turun cukup jauh dan posisi Anda dilikuidasi — jaminan disita dengan diskon, ditambah penalti, permanen, atas pergerakan harga yang mungkin pulih dalam hitungan jam. Likuidasi tidak menunggu Anda bangun.

Jalan keluarnya sudah jelas dari dulu: taruh cadangan, biarkan sesuatu membayar sebagian utang Anda tepat sebelum ambang batas terlewati. Yang menahan orang bukan otomasinya — itu bagian yang gampang, dan **14 tim di hackathon ini sudah membangunnya**. Yang menahan orang adalah pertanyaan yang harus dijawab **sebelum** menyerahkan apa pun:

Apa persisnya yang bisa dilakukan agent ini atas uang saya saat saya tidak menonton, dan bagaimana saya tahu?

Defral menjawabnya dengan menghapus pertanyaannya. Agent kami punya **dua kapabilitas** atas uang Anda. Keduanya **nol argumen** — agent tidak bisa mengirim harga, tidak bisa mengirim jumlah, tidak bisa mengirim tujuan. Keduanya membaca

ulang oracle **di dalam transaksi yang sama** dan **menolak kalau posisi Anda sehat**. Dan uang cadangan Anda **tidak pernah menyingkutkan dompet Anda** — Defral tidak punya fungsi untuk menariknya, karena Defral tidak pernah memegangnya.

Kami tidak meminta Anda mempercayai itu. **Kami menyuruh agent kami mencoba mencuri, di depan Anda, dan menautkan transaksi di mana Base Sepolia menolaknya**. Transaksi gagal tetap transaksi: ia punya hash, ia membakar gas, ia permanen, dan Anda bisa mengkliknya.

Untuk siapa: peminjam yang menjaminkan aset tokenized — US Treasury, emas, aset apa pun yang punya price feed — dan tidak mau memilih antara begadang atau menyerahkan kustodi.

Kriteria penjurian KeeperHub	Bukti kami
Does it work	≥1 transaksi rescue sukses + ≥2 transaksi penolakan, semuanya <code>verified</code> onchain
Would someone use it	Jalur borrower lengkap lewat UI, nol terminal
Kedalaman integrasi	4 tipe trigger, 11 surface KeeperHub, alasan domain per masing-masing (\$13)
Kualitas kode & dokumentasi	Test hijau dari clone bersih · source verified di BaseScan · audit predecessor dipublikasikan penuh (\$5)

2. MASALAH *◀ tulang punggung 1*

Rina meminjam **\$6.000** stablecoin dengan jaminan **10.000 unit** Treasury tokenized yang harganya di par — **\$10.000**. Rasio kesehatannya 166%. Nyaman.

Jumat sore harga jaminannya mulai turun. Bukan crash — drift. Turun 8% dua hari, lalu 12% lagi. Rina tidur. Pukul 03.20 rasio kesehatannya menyentuh **110%**.

Pada titik itu **siapa pun boleh menyita jaminannya**. Bukan sebagian: seorang liquidator mengambil jaminan itu dengan diskon, plus bonus 5%, dan posisi Rina ditutup. Permanen. Empat jam kemudian harganya pulih.

Biaya likuidasi bukan volatilitasnya. Biayanya adalah ketidakhadiran.

Tiga jalan keluar hari ini, dan kenapa ketiganya gagal

Pilihan	Kenapa gagal
Jaga sendiri	Menuntut Rina terjaga sesuai jadwal pasar, bukan jadwalnya. Kegagalan satu malam menghapus keuntungan setahun.
Over-collateralize jauh di atas ambang	Kalau harus menjaminkan \$30.000 untuk pinjam \$6.000, dia sebenarnya tidak sedang meminjam.
Serahkan cadangan ke bot atau vault	Menukar risiko likuidasi dengan risiko counterparty . Bot itu biasanya memegang <code>approve(bot, MAX)</code> — persetujuan tak terbatas. Kalau kuncinya bocor, Rina kehilangan cadangannya tanpa satu pun harga bergerak.

Ke mana lapangan bergerak, dan apa yang mereka lewatkan

Pilihan ketiga yang sedang dikejar semua orang. Di hackathon ini saja ada **14 tim** membangunnya — dua di antaranya bahkan menamai project mereka dengan nama yang sama.

Dan otomasinya memang **bukan bagian yang sulit**. Membaca oracle, membandingkan angka, memanggil `repay` — itu satu sore kerja.

Bagian yang sulit adalah yang tidak satu pun dari 14 tim itu jawab: **Rina harus menyerahkan wewenang atas uangnya kepada sebuah proses yang berjalan saat dia tidur, dan tidak ada yang memberitahunya batas wewenang itu di mana**. Bot-nya sendiri satu-satunya yang memeriksa apakah tindakannya pantas. Kalau kodenya salah, atau kuncinya bocor, atau operatornya berubah pikiran — tidak ada yang menghentikannya.

Itu bukan masalah otomasi. **Itu masalah batas otoritas**, dan itu belum ada yang menyelesaikan.

3. BUKTI

Semua baris di bawah bisa difalsifikasi.

Klaim	Sumber
Nol project RWA di lapangan ini	GitHub search <code>keeperhub + rwa</code> → total_count 0
14 liquidation guardian di window hackathon	14 repo bernama; dua tim sama-sama menamai project mereka bulwark
0 dari 14 membayar dari yield jaminan	Semuanya membayar dari saldo wallet statis
Biaya likuidasi permanen	Diskon + bonus liquidator 500 bps , posisi ditutup
Nol plugin DeFi KeeperHub mendukung testnet	Diverifikasi di 15 halaman plugin
Route protocol-action nol referensi <code>simulate</code>	Issue #1929, <code>grep -c</code> → 0
Sembilan temuan audit atas implementasi kami sendiri	Terverifikasi dari source. Empat yang load-bearing di \$5; lengkapnya Lampiran A

Baris terakhir adalah sumber bukti yang tidak dimiliki tim lain di hackathon ini: **kami sudah membangun versi produk ini sekali, lalu mengaudit kodenya sendiri baris per baris.**

BAGIAN II — APA YANG KAMI SELESAIKAN

4. TESIS: BATAS OTORITAS ◀ *tulang punggung 2*

Tiga klaim. Tidak lebih.

4.1 Agent punya DUA kapabilitas, keduanya nol argumen

```
guardRepay()    ← bayar sebagian utang dari cadangan borrower
sweepCoupon()   ← salurkan yield jaminan ke utang
```

Itu saja. **Tidak ada fungsi ketiga.**

Keduanya **nol argumen**: agent tidak bisa mengirim harga, tidak bisa mengirim jumlah, tidak bisa mengirim penerima. Semua yang menentukan apa yang terjadi **dibaca dari chain di dalam transaksi itu sendiri.**

Dan `guardRepay()` **menolak kalau posisi sehat** — ia membaca ulang oracle, menghitung ulang rasio, lalu `revert` kalau rasionya di atas trigger yang borrower set sendiri.

Yang bertindak adalah agent. Yang memutuskan adalah kontrak.

Pemeriksa keselamatan bukan agent-nya. Pemeriksa keselamatan adalah **callee**-nya — dan callee itu source-nya publik, terverifikasi di BaseScan, dan menolak di depan mata Anda.

4.2 Non-custody: cadangan tidak pernah meninggalkan dompet borrower

```
cadangan = min( saldo dUSD borrower , allowance yang borrower berikan )
```

Mengatur cadangan = memberi `approve` . Menarik cadangan = menurunkan `approve` , atau membelanjakan uangnya.

Konsekuensinya: vault tidak punya `withdraw()` . Tidak punya `topUp()` . Bukan karena belum sempat — **karena tidak ada yang dipegang.**

Baris paling kuat di Capability Matrix kami harfiah benar, dan diverifikasi dengan satu `grep` :

Bisakah agent menarik cadangan Anda? — Tidak ada fungsinya. Kami tidak pernah memegangnya.

Batasnya tajam dan jujur: **jaminan** Anda di-escrow — itu memang arti menjaminkan, tanpa itu bukan pinjaman. **Cadangan pertahanan** Anda tidak.

4.3 Defence Window

```
16667 bps ← posisi Rina di awal ($10.000 jaminan, $6.000 utang)
|
13000 bps ← Guard Trigger – borrower yang set. Agent bertindak DI BAWAH ini.
|
| ← DEFENCE WINDOW · 2000 bps
|     Seluruh tugas cadangan adalah menjaga Rina keluar dari sini.
|
11000 bps ← LIKUIDASI – siapa pun boleh menyita
```

"Pinjam 100, jaminan 110" ITU 11000 basis points.

Model mental tim, dirender persis. Jarak 2000 bps antara trigger dan likuidasi adalah **seluruh ruang kerja cadangan**. Di atas trigger, agent tidak melakukan apa-apa dan tidak boleh melakukan apa-apa. Di bawah 11000, sudah terlambat.

Parameter kanonik

Dirujuk seluruh dokumen. Angka ini muncul identik di Solidity, agent, dan UI — **itu argumennya**.

Parameter	Nilai
<code>LIQUIDATION_BPS</code>	11000 — permissionless
<code>LIQUIDATION_BONUS_BPS</code>	500
Guard Trigger default	13000 (<i>borrower boleh ubah, dibatasi 12000-15000</i>)
Target restore	14500
Defence window	2000 bps
Cadangan	<code>min(balanceOf, allowance)</code>
Fungsi agent	2 , keduanya nol argumen
Jaminan	ERC20 apa pun yang di- <code>allowCollateral</code>
Chain	Base Sepolia <code>84532</code>

BAGIAN III — APA YANG SUDAH ADA, TERMASUK BUATAN KAMI SENDIRI

5. YANG SUDAH DIBANGUN HARI INI ◀ *tulang punggung 3*

5.1 Cermin-RWA: apa yang dibuktikannya

Tim ini membangun **Cermin-RWA** untuk Build on Canton Hackathon (Encode Club), Juli 2026. Canton Network / Daml. Masih hidup di `cermin-rwa.vercel.app`.

Dua hal terbukti di sana, dan keduanya bertahan:

Privasi party-scoped itu nyata di Canton. Signatory/observer set Daml membuat cadangan borrower dan setiap rescue **secara struktural tidak terlihat** oleh lending pool — bukan dienkrpsi, melainkan tidak pernah dikirimkan ke participant-nya. Itu bekerja persis seperti yang diklaim.

Dan ide intinya menyeberang utuh: agent yang melunasi sebagian utang di dalam defence window, sebelum likuidasi. Rumus health ratio, logika defence, dan struktur keputusannya selamat tanpa perubahan.

5.2 Apa yang ternyata tidak pernah diimplementasikan

Sebelum port ini kami mengaudit Cermin baris per baris. Empat temuan mengubah produknya.

Likuidasi tidak pernah ada. `LastResortDefault` — mekanisme yang seharusnya menyita jaminan — badannya **kosong**. Ia tidak mengambil token, tidak memanggil transfer, tidak menyentuh kunci. Komentarnya sendiri mengakui: "*any real-world collateral seizure is an off-ledger process.*"

Nol jaminan pernah berpindah ke lender, di mana pun, di seluruh codebase.

Escrow-nya bukan escrow. `Lock` cuma **disclosure lock** — ia memberi lender hak *melihat*, bukan hak *menahan*. Dan `UnLock` satu-satunya syaratnya adalah "sedang terkunci"; lender **tidak bisa protes**. Borrower bisa membuka kunci dan mentransfer jaminan yang di-pledge **kapan saja selagi pinjaman hidup**.

Cadangannya tidak punya sumber dana. `vaultDeposit` tidak pernah didebit. `TopUp` tidak mengecek apa pun. Shadow Vault-nya adalah **angka di layar**.

Agent hanya bisa mengurangi utang. `TopUpCollateral` ada, tapi controller-nya borrower — agent tidak bisa memanggilnya, dan fungsi itu tidak membeli apa pun, hanya menjamin token yang sudah dimiliki.

Digabung, keempatnya berarti satu hal: Klaim privasi Cermin **benar tentang penyingkapan dan kosong tentang pinjamannya**. Cermin menyembunyikan rescue — yang membela dari likuidasi yang tidak pernah ada — atas escrow yang tidak menahan apa pun.

Bahaya yang dibela Shadow Vault itu **dinarasikan, bukan diimplementasikan**.

5.3 Temuan portabilitas: benar di Canton, fatal di EVM

Lima hal yang **benar** di Canton dan akan **rusak diam-diam** di EVM. Ini temuan portabilitas, bukan kecerobohan.

Di Canton	Di EVM
Idempotency di-key ke contract-id PriceFeed. Jalan karena UpdatePrice consuming — cid berubah tiap tick harga	Alamat oracle tetap selamanya → key selalu cocok → agent fire sekali lalu diam selamanya. Nol error, nol log.
setInterval(tick, 2000) tanpa overlap guard. Aman karena submit-and-wait ~1 detik	Lewat KeeperHub 10-60 detik → cycle bertumpuk → rescue yang sama disubmit belasan kali
GuardPolicy immutable (nol choice)	Dua kontrol UI yang sudah ter-ship — set trigger, toggle coupon sweep — diam-diam tidak melakukan apa-apa
Kupon dengan rate 0 melanggar ensure amount > 0	Jaminan non-yield seperti emas hard-fail , bukan degrade jadi no-op
SweepToLoan tanpa cap	Kupon > utang → transaksi revert , bukan ter-cap

5.4 Klaim yang kami tarik

README Cermin menulis: *"This is impossible to replicate on a public EVM chain."*

Kalimat itu benar, dan justru karena benar ia tidak ikut pindah. Di Base Sepolia setiap storage slot dan setiap event log terbaca dunia.

Klaim privasi **dihapus, bukan dilemahkan**. Kami tidak menambalnya dengan commitment scheme atau menyinggung ZK yang belum dibangun — mengklaim privasi di chain yang tidak punya privasi adalah cara tercepat kehilangan juri yang membuka BaseScan.

Penggantinya adalah **batas otoritas** (\$4) — dan itu justru **menguat** di chain publik, karena pihak ketiga bisa memverifikasinya sendiri tanpa mempercayai kami.

5.5 Alternatif lain di lapangan

14 liquidation guardian. Mereka menyelesaikan otomasi. Tidak satu pun menyelesaikan batas otoritas — mayoritas memegang persetujuan tak terbatas.

Protokol lending nyata (Aave, Morpho, Compound). Anda **tidak bisa menyisipkan trigger assert ke dalam repay() milik orang lain**. Memakai protokol nyata berarti menghapus gerbangnya. Ditambah: nol plugin DeFi KeeperHub mendukung testnet, dan route protocol-action-nya nol referensi `simulate`.

BAGIAN IV — APA YANG DEFRAL PERBAIKI

6. PETA PERBAIKAN ◀ *tulang punggung 4*

Jantung dokumen ini. **Setiap gap di \$5 muncul di sini.** Yang tidak diperbaiki muncul di \$15 sebagai batasan yang diakui.

#	Cermin	Defral	Req	Bukti
G1	<code>LastResortDefault</code> kosong — nol jaminan pernah pindah	<code>liquidate()</code> permissionless . Gate primer = health < 11000. Jaminan benar-benar berpindah ke liquidator + bonus 500 bps	C1, C2	tx <code>Liquidated</code>
G2	<code>Lock</code> = disclosure, borrower bisa unlock kapan saja	Escrow ERC20 sungguhan . <code>transferFrom</code> ke pool; borrower tidak bisa menariknya selagi utang hidup	C3	<code>test_escrowHolds</code>
G3	Cadangan tanpa sumber dana — angka di layar	Non-custodial berbasis allowance . <code>min(balanceOf, allowance)</code> , ditransfer langsung dari wallet borrower ke pool. Nol <code>withdraw</code>, nol <code>topUp</code>	B1, B2	<code>test_abiSurface</code>
G4	Agent cuma bisa repay; top-up mati	Tetap repay-only, dan itu disengaja — dua jalur (<code>guardRepay</code> bergerbang health, <code>sweepCoupon</code> tanpa gerbang, sesuai desain aslinya)	D1, D4	tx <code>Rescued</code> kind 1 & 2
G5	Idempotency di-key ke cid PriceFeed → mati senyap di EVM	Di-anchor ke <code>roundId</code> oracle yang monoton . Harga baru = identitas observasi baru. Ditegakkan onchain via <code>lastActedRound</code>	E3, A4	<code>test_refuseSameRound</code>
G6	<code>setInterval</code> tanpa overlap guard	Overlap guard eksplisit + poll 15 detik. Bug-nya bisa dinyalakan ulang (<code>OVERLAP_GUARD=off</code>) untuk didemokan	E4	demo video
G7	<code>GuardPolicy</code> immutable → dua kontrol UI mati	<code>setPolicy()</code> hidup , dibatasi 12000-15000 bps. Dua kontrol itu berfungsi lagi	D3	<code>test_setPolicyBounds</code>
G8	Kupon rate 0 → abort	Degrade jadi no-op . Jaminan non-yield → <code>couponDue</code> = 0 → agent tidak pernah memanggilnya	D5	<code>test_couponNoYield</code>
G9	<code>SweepToLoan</code> uncapped → revert	Ter-cap di <code>min(couponDue, debt)</code>	D6	<code>test_couponCapped</code>
G10	Terikat satu instrumen bond (<code>couponRateBps</code> , <code>maturity</code> wajib)	<code>allowCollateral(token, yieldBearing)</code> . Treasury dan emas dilindungi engine yang sama	D7	dua jaminan terdaftar
G11	Privasi party-scoped sebagai diferensiator	Ditarik (\$5.4) . Diganti batas otoritas yang bisa diverifikasi pihak ketiga	A1-A4	2 tx penolakan

Satu hal yang menjadi lebih kuat, bukan lebih lemah. Alasan Daml tidak bisa menyita: token ditandatangani issuer+owner, pool **secara kriptografis tidak bisa memindahkannya** sepihak. Di EVM batasan itu tidak ada. **G1 dan G2 bukan port — itu perbaikan yang lingkungan lamanya tidak bisa hosting.**

7. GOAL, NON-GOAL, METRIK

Goal — outcome, bukan output

1. **Juri bisa memverifikasi batas otoritas agent tanpa mempercayai kami** — dengan mengklik transaksi, bukan membaca README.
2. **Borrower melewati penurunan harga tanpa likuidasi dan tanpa menyerahkan kustodi.**
3. **Satu engine melindungi dua kelas jaminan** yang berbeda sifatnya (yield-bearing dan tidak).

Non-goal — masuk akal, tapi sengaja tidak dikerjakan

Non-goal	Kenapa
Privasi	Tidak selamat pindah ke chain publik. Ditarik terbuka (\$5.4)
Onboarding multi-borrower self-service	Satu demo borrower cukup. Insight multi-tenant selamat di derivasi idempotency key
Integrasi protokol lending nyata	Menghapus gerbangnya (\$5.5)
MEV / private mempool routing	Nyata di KeeperHub, tapi mutually exclusive dengan gas sponsorship dan butuh mainnet
Top-up jaminan sebagai pertahanan	Butuh swap; <code>/execute/swap</code> KeeperHub adalah stub 501

Metrik

Kriteria juri	Metrik	Baseline	Target	Cara ukur
Does it work	Transaksi terverifikasi	0	≥1 rescue + ≥2 penolakan + 1 likuidasi	<code>receipts[].verified</code> di BaseScan
Would someone use it	Jalur borrower tanpa terminal	—	100% langkah lewat UI	Dress rehearsal
Kedalaman integrasi	Tipe trigger dipakai	0	4 dari 6	Daftar workflow
Kualitas	Test hijau dari clone bersih	—	100%	<code>npm test</code> + <code>forge test</code>
Usage	Cycle agent tanpa intervensi manual	0	≥50	Log agent

8. APPETITE & CONSTRAINT

Anggaran: 3 hari kerja (10-12 Agt), 3 orang, masing-masing menyetir satu Claude agent. 13 Agt murni freeze. Waktu tetap, scope variabel.

Constraint keras yang **membentuk** solusi ini, bukan sekadar mengganggunya:

Constraint	Konsekuensi desain
KeeperHub nol verb read	Semua pembacaan lewat viem; KeeperHub murni jalur eksekusi
<code>/execute/swap</code> = stub 501	Pertahanan = repay, bukan top-up jaminan
<code>condition</code> <code>check-and-execute</code> nol field <code>functionArgs</code>	Read leg wajib nol argumen → satu vault per borrower
Nol plugin DeFi mendukung testnet	Kontrak sendiri, bukan protokol nyata
Gas testnet gratis dan disponsori	Mem-broadcast kegagalan murah — dan itu jadi bukti utama kami

9. CARA KERJANYA

Tiga kunci yang terpisah — ini yang bikin demo bukan sandiwara

Kunci	Kekuasaan	Agent punya?
Publisher	Mendorong harga oracle	✗ tidak pernah
Borrower	Set cadangan, set policy, revoke	✗ tidak pernah
Turnkey enclave EOA (<i>KeeperHub</i>)	Dua fungsi agent. Itu saja.	✓ dan hanya ini

Private key ketiga **tidak pernah kami pegang** — KeeperHub yang menyimpannya di secure enclave.

Satu siklus pertahanan

1. Observasi harga baru masuk

→ roundId naik

2. Snapshot atomik satu blok

harga + posisi + cadangan, sekaligus

3. Hitung ulang rasio kesehatan

4. Sehat?

→ tidak ada write. Titik.

5. Di defence window?

→ bayar sebagian utang dari cadangan borrower

LANGSUNG ke pool. Vault tidak menyentuhnya.

6. Catat

→ event onchain: berapa, dari rasio berapa ke berapa, harga berapa, observasi mana

Langkah 4 dan 5 **dijaga di dalam transaksi**, bukan di dalam agent. Kalau agent salah menilai, kontraknya menolak.

Empat workflow, empat alasan domain

	Trigger	Kenapa
W1 guard	Event — harga baru dipublish	Observasi harga adalah eventnya
W2 coupon	Schedule — kalender kupon	Menghasilkan transaksi di hari pasar tidak bergerak
W3 watchdog	Block	Event trigger nol dokumentasi reorg/catch-up. Ini mitigasinya
W4 health	Manual	Read-only. Satu-satunya yang dipublikasikan

10. REQUIREMENT

Definition of Done global — berlaku untuk semua, tidak diulang per baris: `forge test` + `npm test` hijau dari clone bersih · source **verified** di BaseScan · alamat dipublikasikan · setiap response eksekusi diarsipkan sebagai JSON dan **di-commit**.

Prioritas: **Must** · **Should** · **Could**. *Must* maksimum 60% effort.

A — Batas otoritas agent · owner: alven

#	Requirement	P	Acceptance	Traces
A1	Agent punya tepat dua fungsi, keduanya nol argumen	M	Fuzz test: nol fungsi <code>onlyAgent</code> menerima <code>address</code> atau <code>uint256</code>	\$4.1
A2	<code>guardRepay()</code> revert saat rasio $\geq$ trigger	M	<i>Given</i> posisi sehat <i>When</i> agent memanggil <i>Then</i> revert <code>Refused_Healthy</code>	\$2, G11
A3	<code>guardRepay()</code> revert saat oracle basi > 1 jam	M	<i>Given</i> <code>updatedAt</code> lewat batas <i>Then</i> revert <code>Refused_StaleOracle</code>	\$4.1
A4	<code>guardRepay()</code> revert pada observasi harga yang sudah ditindak	M	Dua panggilan pada <code>roundId</code> sama → yang kedua revert	G5
A5	Borrower bisa mencabut agent kapan saja, sepihak	M	Setelah <code>revokeAgent()</code> , semua panggilan agent revert	\$2
A6	Nol proxy, nol <code>delegatecall</code> , nol upgrade path, nol <code>owner</code> di vault	M	Pembacaan source + ABI surface test	\$4.1

B — Non-custody · owner: alven

#	Requirement	P	Acceptance	Traces
B1	Vault tidak punya <code>withdraw()</code> maupun <code>topUp()</code>	M	ABI tidak memuat keduanya	G3, \$4.2
B2	Cadangan = <code>min(balanceOf, allowance)</code> ; dana bergerak langsung dari wallet borrower ke pool	M	Saldo vault nol sebelum dan sesudah rescue	G3
B3	Saldo token EOA agent persis nol setelah demo	M	Read Contract di BaseScan, on camera	\$4.2

C — Likuidasi & escrow nyata · owner: alven

#	Requirement	P	Acceptance	Traces
C1	<code>liquidate()</code> bisa dipanggil siapa pun saat rasio < 11000	M	Wallet acak berhasil melikuidasi posisi tak terjaga	G1
C2	Likuidasi benar-benar memindahkan jaminan + bonus 500 bps	M	Saldo jaminan liquidator naik; event <code>Liquidated</code>	G1
C3	Jaminan di-escrow ; borrower tidak bisa menariknya selagi utang hidup	M	<code>withdrawCollateral</code> revert saat <code>debt > 0</code>	G2
C4	Grace period menunda likuidasi, tapi health tetap gerbang primer	S	Posisi sehat tidak bisa dilikuidasi walau ada grace kedaluwarsa	G1

D — Defence engine · owner: alven + islakun

#	Requirement	P	Acceptance	Traces
D1	Jumlah bayar dihitung callee : <code>min(kebutuhan, cap, cadangan)</code>	M	Agent tidak mengirim jumlah; tiga cap semuanya ada test-nya	G4
D2	Rasio & jumlah bayar identik di Solidity, agent, dan UI	M	16667 · 12667 · 758.62 · 5241.38 · 14500 di tiga tempat	\$4.3
D3	Borrower bisa mengubah policy, dibatasi 12000-15000 bps	M	Di luar batas → revert. Dua kontrol UI berfungsi	G7
D4	<code>sweepCoupon()</code> tanpa gerbang health — paydown proaktif	M	Berhasil pada posisi sehat	G4
D5	Jaminan non-yield → <code>couponDue = 0</code> , no-op bukan abort	M	Emas terdaftar; agent tidak pernah memanggil sweep	G8
D6	Sweep ter-cap di <code>min(couponDue, debt)</code>	M	Kupon > utang → ter-cap, tidak revert	G9
D7	<code>allowCollateral(token, yieldBearing)</code> ; dua jaminan terdaftar	M	Treasury + emas, engine sama	G10

E — Eksekusi & keandalan lewat KeeperHub · owner: islakun

#	Requirement	P	Acceptance	Traces
E1	Setiap perubahan state agent lewat KeeperHub	M	Nol jalur bypass di codebase	\$1
E2	Transaksi ter-revert diterjemahkan jadi kegagalan , bukan sukses	M	KeeperHub balik HTTP 200 pada revert; agent harus throw	—
E3	Hasil tidak diketahui (<code>timeout</code> / <code>not_found</code> / <code>unconfirmed</code>) tidak pernah memicu broadcast ulang	M	Direkonsiliasi lewat event onchain sendiri	G5
E4	Nol cycle bertumpuk	M	<code>OVERLAP_GUARD=off</code> mendemokan bug-nya; onchain menangkapnya	G6
E5	Retry membedakan kegagalan deterministik dari kegagalan infra	S	Revert dengan alasan → tidak retry. Error jaringan → retry	G5
E6	Empat tipe trigger dipakai, masing-masing beralasan domain	S	Empat workflow aktif	\$9

F — Bukti & artefak · owner: islakun + bima

#	Requirement	P	Acceptance	Traces
F1	Satu perintah menghasilkan seluruh rantai bukti	M	<code>prove.ts</code> , nol argumen, dijalankan live	\$1
F2	≥2 transaksi penolakan yang sengaja di-broadcast, <code>verified</code>	M	<code>receiptStatus: reverted</code> , <code>verified: true</code>	\$4.1
F3	Demo memuat posisi tak terjaga yang benar-benar dilikuidasi	M	Dua tx berdampingan, harga sama, hasil berbeda	G1
F4	Capability Matrix: tiap baris punya tx nyata atau pernyataan fungsinya tidak ada	M	Nol baris tanpa bukti	\$4.2
F5	Semua bukti diarsipkan sebagai JSON ter-commit	M	Halaman bukti hidup walau API mati	\$16

11. FLOW & PERILAKU EDGE CASE

Jalur borrower

- 1. Buka posisi — jaminan aset, pinjam stablecoin (*escrow nyata*)
- 2. Set cadangan — `approve` sejumlah yang mau dipertaruhkan (*uangnya tetap di wallet*)
- 3. Set Guard Trigger — default 13000 bps
- 4. **Tidur**
- 5. Harga turun → agent bertindak → notifikasi masuk
- 6. Kapan saja: cabut agent, ubah policy, atau turunkan allowance

Perilaku saat tidak normal

Keadaan	Perilaku
Posisi sehat	Nol write. Revert dengan alasan eksplisit kalau dipaksa
Oracle basi	Revert. Harga benar pun ditolak kalau terlalu tua
Observasi harga sama	Revert. Satu observasi, satu tindakan
Cadangan nol	Buka grace period, beri tahu pool. Bukan rescue diam-diam
Cadangan < kebutuhan	Bayar sebisanya. Sebagian lebih baik dari nol
Agent dicabut	Semua panggilan agent revert. Uang cuma bisa bergerak ke borrower
Jaminan tanpa yield	Sweep tidak pernah dipanggil. No-op, bukan error
Kupon > utang	Ter-cap di utang
Cycle bertumpuk	Guard di agent dan di kontrak
Hasil eksekusi tak diketahui	Tidak pernah broadcast ulang. Rekonsiliasi lewat event sendiri
Dua implementasi rasio menyimpang	Blokir write , alert

12. ALTERNATIF YANG DIPERTIMBANGKAN

Alternatif	Kenapa kalah
Pakai protokol lending nyata	Tidak bisa menyisipkan trigger assert ke <code>repay()</code> orang lain → menghapus gerbangnya. Plus nol plugin DeFi bertestnet
Cadangan custodial (<code>topUp</code> / <code>withdraw</code>)	Lebih mudah, tapi mematikan baris terkuat kami: <i>"tidak ada fungsinya — kami tidak pernah memegangnya"</i>
Kontrak monolitik , <code>guardRepay(address, uint80)</code>	Lebih sedikit deploy, tapi mematikan nol-argumen — dan nol-argumen syarat <code>check-and-execute</code>
Top-up jaminan sebagai pertahanan	Butuh swap; <code>/execute/swap</code> stub 501. Dan itu taruhan arah, bukan pertahanan
Private mempool routing	Nyata, tapi mutually exclusive dengan gas sponsorship dan butuh mainnet. Dilaporkan sebagai temuan, bukan dibangun
LLM menyusun calldata (<code>mcp:write</code>)	Menaruh LLM di jalur yang memindahkan uang mengontradiksi tesis kami sendiri

Retraksi: klaim privasi Daml (\$5.4). Bukan karena salah — karena lingkungannya berubah, dan klaim itu tidak selamat.

13. KEDALAMAN INTEGRASI KEEPERHUB

Section bernama karena ini 1 dari 4 kriteria penjurian.

Dipakai: eksekusi contract-call · `check-and-execute` (gerbang ganda: KeeperHub menolak **dan** kontrak menolak) · preflight simulate · status polling yang menghormati hint interval · Idempotency-Key yang di-anchor ke observasi harga · **empat tipe trigger** · batch read via Multicall3 (snapshot atomik) · query contract events · run code · notifikasi Discord di **kedua** cabang kondisi · CLI · MCP read-only.

Sengaja tidak dipakai, dan alasannya:

Alasan	
<code>mcp:write</code>	Menaruh LLM di jalur pemindah uang mengontradiksi tesis kami
<code>/execute/transfer</code>	Bentuk query param <code>simulate</code> diabaikan (#1959). Nol panggilan di seluruh repo
<code>/execute/node</code>	Endpoint terkuat platform, tapi hanya terverifikasi dari branch staging
Plugin DeFi	Nol testnet, nol <code>simulate</code> , menghapus gerbang

Yang TIDAK kami klaim: nol klaim tentang exponential backoff · nol klaim MEV protection · nol klaim endpoint audit-trail platform. **Audit trail kami milik kami sendiri**, dibangun di atas status eksekusi dan event kami sendiri.

Kejujuran tentang batas platform adalah bagian dari kedalaman, bukan pengurangannya.

14. RISIKO & PERTANYAAN TERBUKA

Risiko

#	Risiko	Mitigasi	Owner	Kapan
R1	Nol Solidity ada saat mulai; tim terakhir menulis Daml	Deploy stub dengan ABI beku di jam ke-3, sebelum logika ditulis	alven	Min 10
R2	🔴 "Kalian yang menulis pool, oracle, dan mendorong harganya sendiri — membuktikan apa?"	Artefak, bukan slide: tiga kunci terpisah dinyatakan di muka · dua tx penolakan · saldo EOA agent nol	semua	\$17
R3	Ini terbaca sebagai liquidation guardian ke-15	Nol mitigasi teknis. 20 detik pertama video + paragraf pertama README	bima	Sel 12
R4	Log KeeperHub kedaluwarsa sebelum penjurian 17-20 Agt	Arsip JSON ter-commit sejak hari pertama	islakun	Min 10

Pertanyaan terbuka — *impact tinggi, confidence rendah, di atas*

#	Pertanyaan	Kalau jawabannya buruk	Diprobe
Q1	<code>msg.sender</code> di bawah gas sponsorship — EOA enclave atau relay?	Seluruh tesis perlu ditulis ulang jadi allowlist, dan klaim diturunkan terbuka	Min menit 50
Q2	Apakah KeeperHub mem-broadcast transaksi yang pasti revert?	F2 dan F3 hilang — bukti penolakan harus didesain ulang	Min menit 65
Q3	Apakah <code>check-and-execute</code> menerima view nol-argumen?	Turun ke contract-call. Downgrade, bukan kill	Min menit 80
Q4	Retensi log free-tier sampai 17-20 Agt?	Sudah dimitigasi oleh R4	Min 10

Gate keputusan

	Kapan	Lolos kalau
CP1	Min menit 50	Q1 terjawab
CP2	Min menit 65	Q2 terjawab
CP3	Min menit 150	ABI beku + stub ter-deploy ← paling mahal kalau lewat
CP4	Sen 11.00	Kontrak asli ter-deploy + verified
CP5	Sen 15.00	Suite test lulus tanpa diubah
CP6	Sen 21.00	Rescue nyata pertama

Kalau B6 lewat, korbankan berurutan sampai video hari Selasa aman: x402 listing → differential oracle → W3+W4 → reconciler → W1. **Jangan pernah geser tanggal video.**

15. TIDAK DIKERJAKAN

Tidak siklus ini: privasi · multi-borrower onboarding · integrasi protokol lending nyata · private mempool routing · top-up jaminan · differential fuzz-grid lintas implementasi.

Batasan yang diakui di muka — dinyatakan supaya juri tidak menemukannya sendiri:

- **Satu instrumen per posisi.** Emas *dan* Treasury menjamin satu posisi yang sama tidak bisa diekspresikan.
- **Oracle publisher-pushed.** Untuk NAV RWA ini jujur, bukan shortcut — NAV memang dipublikasikan, bukan diturunkan dari DEX. Berbentuk `AggregatorV3Interface` supaya jalur mainnet hanya ganti satu alamat.
- **Pool dan oracle kami tulis sendiri.** Dijawab langsung di §17.
- **Grace period** ada sebagai state, bukan sebagai mesin 72 jam penuh.

16. RILIS, GATE, DAN BUKTI

Jadwal per jam ada di tiga `plan.md`. Di sini hanya yang mengikat lintas *workstream*.

Ketergantungan:

alven: probe → FREEZE ABI + stub
(Min jam 3)

islakun: agent → rescue pertama → prove.ts

bima: UI → halaman bukti → VIDEO

Freeze ABI di jam ke-3 memutus rantai serial dari ~26 jam jadi ~3 jam. Setelah itu **nol orang mengganggu**.

Instrumentasi bukti = requirement, bukan kebiasaan. Setiap response eksekusi diarsipkan sebagai JSON dan di-commit sejak hari pertama. Transaksi kami 11-12 Agt; penjurian 17-20 Agt; retensi log free-tier tidak terverifikasi. **Halaman bukti membaca arsip statik, bukan API live.**

Aturan video yang mengikat: nol beat menampilkan UI tanpa transaksi di sebelahnya. Tiap klaim berakhir di BaseScan dalam sepuluh detik. Beat yang tidak bisa — dipotong. (*Shot list: FE/`plan.md` §5.*)

Freeze 13 Agt: nol commit kode. Buka ulang tiap tautan, konfirmasi video diputar dari incognito, konfirmasi repo publik.

Mulai dari mana: baca `plan.md` sesuai peran Anda. alven mulai dari TASK 0 — dua orang lain menunggu ABI itu.

17. FAQ

Eksternal

Apa yang terjadi kalau saya tidak punya cadangan saat harga turun? Agent membuka grace period dan memberi tahu pool. Ia tidak berpura-pura menyelamatkan Anda.

Bisa saya pakai emas, bukan Treasury? Ya. Jaminan apa pun yang punya price feed. Yang berbeda cuma satu: emas tidak membayar kupon, jadi fitur "pinjaman melunasi dirinya sendiri" tidak menyala. Bukan error — memang tidak berlaku.

Berapa yang bisa diambil agent dari saya? Maksimum per kejadian yang Anda set, dibatasi kebutuhan untuk kembali sehat, dibatasi cadangan Anda. Dan **hanya** ke lending pool — tidak ada fungsi untuk mengirimnya ke tempat lain.

Internal

● **Kalian yang menulis pool-nya, oracle-nya, dan kalian sendiri yang mendorong harganya. Kalian membuktikan apa?**

Yang kami buktikan bukan bahwa harganya turun — itu memang kami yang lakukan. Yang kami buktikan adalah **apa yang tidak bisa dilakukan agent saat harga turun**. Tiga kunci terpisah: kunci publisher yang menggerakkan harga bukan milik agent; kunci borrower yang mendanai cadangan bukan milik agent; kunci agent hanya bisa memanggil dua fungsi dan **kami tidak pernah memegang private key-nya** — KeeperHub yang menyimpannya di secure enclave.

Buktinya artefak, bukan pernyataan: **dua transaksi di mana agent kami mencoba dan chain menolak**, ditambah saldo token agent **persis nol** setelah memindahkan uang orang lain melewati gerbang yang tidak bisa ia lebarkan.

Kenapa kami harus percaya versi ini benar-benar mengimplementasikan apa yang versi lama cuma narasikan?

Karena Anda bisa mengklik likuidasinya. Di Cermin, likuidasi tidak pernah terjadi — badannya kosong (\$5.2). Di Defral, demo menjalankan **dua posisi** dengan harga yang sama: satu dijaga, satu tidak. Yang tidak dijaga **kehilangan jaminannya**, dan transaksinya ada.

Ini liquidation guardian ke-15. Kenapa berbeda?

14 yang lain mengirimkan **otomasinya**. Kami mengirimkan **batas otoritasnya**, beserta harness yang memfalsifikasinya. Mereka minta juri percaya agent mereka berperilaku baik; kami mempersilakan juri menyuruh agent kami mencoba mencuri.

Kalau kunci agent bocor malam ini, apa yang hilang?

Penyerang bisa memanggil dua fungsi nol-argumen. Keduanya membaca ulang oracle dan menolak kalau posisi sehat. Cadangan tetap di wallet borrower. Yang bisa dicapai penyerang: **membayar utang orang lain lebih awal**. Bandingkan dengan pola `approve(bot, MAX)` yang dominan di lapangan ini.

Kenapa klaim privasi kalian hilang?

Karena benar, dan karena itu tidak selamat. Detail di §5.4. Kami menghapusnya alih-alih melemahkannya, dan menurut kami itu poin kredibilitas, bukan kehilangan.

Kenapa mem-broadcast transaksi yang pasti gagal?

Hasil simulasi tidak meninggalkan jejak apa pun — ia bukan bukti. **Transaksi ter-revert punya hash, membakar gas, permanen, dan bisa diverifikasi pihak ketiga.** Gas testnet gratis, jadi penolakan bisa dibuktikan dengan harga nol. Itu satu-satunya alasan kami membakar gas untuk gagal.

Apa yang nyata dan apa yang mock?

Nyata: kontrak, escrow, transfer token, likuidasi, semua transaksi, eksekusi lewat KeeperHub, test suite. Mock: pool lending, oracle, dua token jaminan — semuanya kami deploy, dinyatakan terbuka, dan alasannya di §8 dan §15.

Apa yang tidak selesai?

Apa pun yang dikorbankan di gate CP6 (§14) dicatat di README pada malam submission, dengan nama dan alasan. Kami tidak menyembunyikan yang dipotong.

LAMPIRAN

A. Sembilan temuan audit Cermin — lengkap, dengan sitasi file dan baris. **B. Parameter kanonik & angka demo** — 16667 · 12667 · 758.62 · 5241.38 · 14500 · 112.50 · 5128.88 · 14818. Catatan: pembulatan **half-away-from-zero**; Solidity harus memakai $(a \cdot b + d/2)/d$, karena truncate biasa menghasilkan 12666 dan memutuskan argumen "satu angka di tiga implementasi". **C. Peta surface & gotcha KeeperHub** — kanonik di `SC/plan.md` §7 dan `BE/plan.md` §6. Tidak disalin ke sini. **D. Bounty \$1.000** — workstream terpisah. **Tidak boleh menyandera submission.** **E. Glossary** — *defence window · reserve · escrow · roundId · refusal receipt*.